

RAMRAO ADIK INSTITUTE OF TECHNOLOGY

D Y PATIL DEEMED TO BE UNIVERSITY, NERUL, NAVI MUMBAI
Department of Computer Engineering | Academic Year 2026 – 2027

LABORATORY PRACTICAL JOURNAL SUBMISSION

Skill Based Lab - Advanced Web Technology (SBL - AWT)

Comprehensive Input & Output Specifications Manual Experiments 01 to 10

Student Name:	Asmit Jogdand
Roll Number:	25CE1051
Class & Branch:	B.Tech Computer Engineering (Sem VI)
Problem Statement:	HealthPulse Digital Healthcare Portal
GitHub Repository:	github.com/SmitroniX/SBL-EXPT
Total Experiments:	10 / 10 Fully Documented & Verified

Certified that this is the bonafide record of laboratory work performed by Asmit Jogdand (25CE1051) during the academic year 2026 – 2027 in the Advanced Web Technology Laboratory.

Skill Based Lab - Advanced Web Technology (SBL-AWT)

Master Input & Output Journal (Experiments 01 – 10)

Student Profile & Academic Credentials

- **Student Name:** Asmit Jogdand
- **Roll Number:** 25CE1051
- **Class / Division:** B.Tech Computer Engineering (Sem VI)
- **Academic Year:** 2026 – 2027
- **Course:** Skill Based Lab - Advanced Web Technology (SBL-AWT)
- **Institution:** Ramrao Adik Institute of Technology (RAIT), D Y Patil Deemed to be University, Nerul, Navi Mumbai
- **Problem Statement Domain:** HealthPulse — Digital Healthcare & Clinical Management Portal
- **Repository URL:** <https://github.com/SmitroniX/SBL-EXPT>

Master Index of Laboratory Experiments

Expt. No.	Official Syllabus Title	Implementation Focus (HealthPulse)	Input / Output Manual	Status
01	Design a Registration Form using HTML5 and CSS3	Patient Health Profile & HIPAA Insurance Registration	View Manual	VERIFIED
02	Password Strength Indicator & Star Rating System using jQuery	Staff Credential Entropy & Doctor Consultation 5-Star Rating	View Manual	VERIFIED
03	Design a Homepage using React.js	Multi-Specialty Clinical Homepage with Live Department Filtering	View Manual	VERIFIED
04	Create a Simple Login Form using React.js	Multi-Role Authentication Portal with Session Dashboard	View Manual	VERIFIED
05	Connecting Your React.js Project with MongoDB	Full-Stack MERN Appointment Hub with Resilient In-Memory Fallback	View Manual	VERIFIED
06	Design a Web Page using Node.js	Pure Node.js Server with Streaming & System Telemetry	View Manual	VERIFIED
07	Design a Form Using Express.js in Node.js	Specialist Consultation Form with Server-side Validation & Receipt	View Manual	VERIFIED
08	Integration of Third-Party Services	Clinical Prescriptions (Email), SMS OTP (Twilio), Fees (Razorpay)	View Manual	VERIFIED
09	Hosting Website with Domain Registration Process	Cloud Hosting Guide & DNS Zone Architecture (healthpulse-care.org)	View Manual	VERIFIED
10	Integration of SSL Certificate in Web Application	Dual HTTPS Server (8443) & HTTP 301 Redirection (8080)	View Manual	VERIFIED

Experiment 01: Design a Registration Form using HTML5 and CSS3

- **Directory:** `Expt-01-Registration-Form`
- **Detailed Manual:** `Expt-01-Registration-Form/INPUT_OUTPUT.md`

1.1 Input Specification

- **Files:** `index.html`, `style.css`, `script.js`
- **Execution Command:** `bash cd Expt-01-Registration-Form && chromium-browser index.html`
- **Validation Constraints:**
- **Name:** `pattern="^[a-zA-Z\s]+$"`, `minlength="3"` (Alphabets only)
- **DOB:** `max="2026-01-01"` (Restricts future birthdates)
- **Phone:** `pattern="[6-9][0-9]{9}"` (10-digit Indian cellular number)
- **Email:** `type="email"` (RFC standard email formatting)
- **Password:** `(?=.*\d)(?=.*[a-z])(?=.*[A-Z])(?=.*[\W]).{8,}` (8+ chars, uppercase, digit, symbol)
- **Document Upload:** `accept=".pdf, .png, .jpg, .jpeg"`
- **Consent:** `required` checkbox
- **Sample Input Vector (Valid):**

```
json { "fullName": "Asmit Jogdand", "dob": "2004-08-15", "gender": "Male", "bloodGroup": "O+", "phone": "9876543210", "email": "asmit.jogdand@healthpulse.org", "emergencyContact": "9820112233", "address": "Flat 402, Seawoods Grand Central, Nerul, Navi Mumbai", "chronicConditions": ["Hypertension", "Asthma"], "insurancePolicyNo": "POL-992144-HP", "accountPassword": "HealthPulse@2026!", "confirmPassword": "HealthPulse@2026!", "hipaaConsent": true }
```

1.2 Output Specification

- **Rendered Graphical Interface:** Clinical blue theme card featuring grouped fieldsets, floating input labels, and interactive validation styling.
- **Dynamic Confirmation Modal:**

```
+-----+ | o PATIENT
ENROLLMENT CONFIRMED |
+-----+ | HealthPulse Record
Identifier: HP-2026-992144 | | Patient Name: Asmit Jogdand (DOB: 15/08/2004, Blood: O+) | | Registered
Contact: +91-9876543210 | asmit.jogdand@healthpulse.org | | Insurance Policy: Star Health Allied Insurance
(POL-992144-HP) | | Status: Active Patient Profile Verified |
+-----+
```

- **Test Case Result:** Rejected numeric names (`12345`) and password mismatches; passed full valid submission. Status: **PASS.**

Experiment 02: Password Strength Indicator & Star Rating System using jQuery

- **Directory:** `Expt-02-Password-Rating-jQuery`
- **Detailed Manual:** `Expt-02-Password-Rating-jQuery/INPUT_OUTPUT.md`

2.1 Input Specification

- **Files:** `index.html`, `style.css`, `script.js` (jQuery v3.7.1)
- **Execution Command:** `bash cd Expt-02-Password-Rating-jQuery && chromium-browser index.html`
- **Rules Evaluated on input keyup** : 1. Length ≥ 8 chars (`val.length >= 8`) 2. Lowercase letter (`/[a-z]/.test(val)`) 3. Uppercase letter (`/[A-Z]/.test(val)`) 4. Numeric digit (`/[0-9]/.test(val)`) 5. Special symbol (`/[!@#$%^&*()_+~\-=\[\]{}';":\|,.<>\/?]/.test(val)`)
- **Doctor Review Input:**
 - Selected Physician: `Dr. Amarsinh V. Vidhate (Chief Medical Director - Cardiology)`
 - Star Clicked: `Star 5 ( data-value="5" )`
 - Feedback: `"Dr. Vidhate provided prompt and thorough cardiac diagnostics. Highly recommended."`

2.2 Output Specification

- **Real-Time Password Entropy Checklist & Meter:**
 - `adm` $\rightarrow$ 20% (Very Weak, Red `#ef4444`)
 - `HealthPulse` $\rightarrow$ 60% (Medium, Yellow `#eab308`)
 - `HealthPulse@2026!` $\rightarrow$ 100% (Very Strong, Emerald `#10b981`)
- **Doctor Consultation Submission Output (jQuery `.fadeIn()`):**

```
+-----+ | DOCTOR
CONSULTATION REVIEW SUBMITTED SUCCESSFULLY |
+-----+ | Consulting
Specialist: Dr. Amarsinh V. Vidhate (Chief Medical Director) | | Awarded Rating: ***** 5.0 / 5.0
(Exceptional Experience) | | Patient Feedback: "Dr. Vidhate provided prompt and thorough cardiac | |
diagnostics. Highly recommended." | | Status: Published to Clinical Quality Board |
+-----+
```
- **Test Case Result:** All 5 regex criteria and star interactions verified. Status: **PASS**.

Experiment 03: Design a Homepage using React.js

- **Directory:** `Expt-03-React-Homepage`
- **Detailed Manual:** `Expt-03-React-Homepage/INPUT_OUTPUT.md`

3.1 Input Specification

- **Files:** `index.html`, `package.json` (React 18, Babel Standalone)
- **Execution Command:** `bash cd Expt-03-React-Homepage && chromium-browser index.html`
- **Component Hierarchy:** `App` $\rightarrow$ `StudentHeader`, `Navbar`, `HeroSection` (`StatsBar`), `ServicesSection` (`FilterTabs`, `ServiceCards`), `SpecialistSection`, `EmergencyBanner`, `Footer`.
- **User Actions Injected:**
 - Category Filter Clicks: `"All"`, `"Critical Care"`, `"Outpatient"`, `"Diagnostics"`
 - Appointment Trigger: Click `"Book Appointment"` button

3.2 Output Specification

- **Rendered Graphical Interface:** Full healthcare portal displaying key metrics (45,000+ Patients, 120+ Specialists, 24/7 Trauma Care, 99.8% Recovery Rate).
- **Virtual DOM State Filtering Output:**
 - Selecting `"Critical Care"` instantly displays only Cardiology & Neurology without reloading the webpage.
 - Selecting `"Diagnostics"` filters to Pathology and 3T MRI & CT.
- **Booking Output:** `Generated Queue Token: HP-APPT-88219 | Slot Reserved: Cardiology & CathLab (Dr. Vidhate)`
- **Test Case Result:** Category state transitions verified cleanly. Status: **PASS**.

Experiment 04: Create a Simple Login Form using React.js

- **Directory:** `Expt-04-React-Login-Form`
- **Detailed Manual:** `Expt-04-React-Login-Form/INPUT_OUTPUT.md`

4.1 Input Specification

- **Files:** `index.html`, `package.json` (React 18)
- **Execution Command:** `bash cd Expt-04-React-Login-Form && chromium-browser index.html`
- **Controlled State Hooks:** `activeRole`, `identifier`, `password`, `showPassword`, `isLoading`, `isAuthenticated`, `currentUser`.
- **Profiles Injected:**
 - Patient: `patient@healthpulse.org` / `Patient@1234`
 - Doctor: `dr.vidhate@healthpulse.org` / `Doctor@5678`
 - Admin: `admin@healthpulse.org` / `Admin@9999`

4.2 Output Specification

- **Authentication Feedback:** Spinner displays "Authenticating Session..." with simulated 600ms latency.
- **Conditional Rendering Output (Clinical Session Dashboard):**

```
+-----+ |
HEALTHPULSE CLINICAL SESSION DASHBOARD Session ID: hp_jwt_77a91bf2 |
+-----+ |
Welcome back, Asmit Jogdand! Role: [ ▣ PATIENT PORTAL ACCESS ] | | Active Medical ID: HP-2026-9921
Authenticated At: 2026-10-01 10:25:00 IST | | Quick Actions: [ View Lab Reports ] [ Active Prescriptions ]
[ Upcoming Consultations ] |
+-----+
```

- **Test Case Result:** Validation against short passwords (<6 chars) and role switching verified. Status: **PASS**.

Experiment 05: Connecting React.js Project with MongoDB

- **Directory:** [Expt-05-React-MongoDB-Connect](#)
- **Detailed Manual:** [Expt-05-React-MongoDB-Connect/INPUT_OUTPUT.md](#)

5.1 Input Specification

- **Files:** `server.js`, `package.json`, `public/index.html` (MERN Stack)
- **Execution Command:** `bash cd Expt-05-React-MongoDB-Connect && npm install && PORT=5000 node server.js`
- **POST Appointment Request Payload:**
`bash curl -X POST http://localhost:5000/api/appointments \ -H "Content-Type: application/json" \ -d '{ "patientName": "Rohan Deshmukh", "doctorName": "Dr. Amarsinh V. Vidhate", "department": "Cardiology", "appointmentDate": "2026-10-05", "timeSlot": "11:00 AM", "priority": "Urgent" }'`

5.2 Output Specification

- **Server Execution Output:** `[HealthPulse] Server listening on http://localhost:5000 [MongoDB] Local MongoDB offline -> Transparently activating In-Memory Resilient Store.`
 - **API Insertion Response:** `json HTTP/1.1 201 Created { "success": true, "message": "Appointment saved in In-Memory store", "data": { "_id": "apt_1790847066167", "patientName": "Rohan Deshmukh", "doctorName": "Dr. Amarsinh V. Vidhate", "department": "Cardiology", "appointmentDate": "2026-10-05", "timeSlot": "11:00 AM", "priority": "Urgent", "status": "Confirmed" } }`
 - **Test Case Result:** GET, POST, and DELETE endpoints verified with zero downtime fallback. Status: **PASS**.
-

Experiment 06: Design a Web Page using Node.js (Pure Core Modules)

- **Directory:** `Expt-06-NodeJS-WebPage`
- **Detailed Manual:** `Expt-06-NodeJS-WebPage/INPUT_OUTPUT.md`

6.1 Input Specification

- **Files:** `server.js`, `package.json` (Zero npm dependencies; pure `http`, `url`, `os`, `path`)
- **Execution Command:** `bash cd Expt-06-NodeJS-WebPage && PORT=3555 node server.js`
- **Client POST Request:** `bash curl -X POST http://localhost:3555/consultation \ -d "fullName=Asmit+Jogdand&email=asmit%40healthpulse.org&phone=9876543210&department=Cardiology&symptoms=Routine+Checkup"`

6.2 Output Specification

- **Server Startup & Request Logs:** `[HealthPulse] Native Node.js server is running at http://localhost:3555`
`[HealthPulse] Student: Asmit Jogdand (25CE1051) [HTTP GET] /api/status -> 200 OK [HTTP POST] /consultation -> Stream buffer parsed -> Issued ticket HP-253968`
- **JSON Telemetry Endpoint (GET /api/status):** `json { "status": "ONLINE", "application": "HealthPulse Node Server", "developer": "Asmit Jogdand (25CE1051)", "nodeVersion": "v24.20.0", "memory": { "free": "4504.99 MB", "total": "11927.42 MB" }, "uptimeSeconds": 42 }`
- **Dynamic Ticket Output:** Generated HTML pass with booking reference `HP-253968`. Status: **PASS**.

Experiment 07: Design a Form Using Express.js in Node.js

- **Directory:** `Expt-07-ExpressJS-Form`
- **Detailed Manual:** `Expt-07-ExpressJS-Form/INPUT_OUTPUT.md`

7.1 Input Specification

- **Files:** `server.js`, `package.json` (Express.js)
- **Execution Command:** `bash cd Expt-07-ExpressJS-Form && npm install && PORT=4000 node server.js`
- **Validation Rules:**
 - Name \$lge 3\$ characters
 - Mobile phone: regex `/^[6-9]\d{9}$/`
 - Valid RFC email format
 - Consulting physician selected
- **Pricing Logic:** Telehealth ₹500, OPD ₹800, Home Visit ₹1,500
- **Test Request:** `bash curl -X POST http://localhost:4000/consultation \ -d "patientName=Asmit+Jogdand&contactNumber=9876543210&emailAddress=asmit%40healthpulse.org&specialist=Dr.+Amarsinh+V.+Vidhate+(Cardiology)&visitType=Telehealth"`

7.2 Output Specification

- **Validation Failure State (HTTP 400 Bad Request):**
 - Renders error banner with highlighted inputs and preserved user data.
- **Validation Success State (HTTP 200 OK):**

```
+-----+ | HEALTHPULSE
MEDICAL CLINIC – OFFICIAL CONSULTATION RECEIPT |
+-----+ | Consultation
Booking ID: HP-CONS-882194 | | Patient Legal Name: Asmit Jogdand (+91-9876543210) | | Consulting Physician:
Dr. Amarsinh V. Vidhate (Cardiology) | | Visit Mode & Fee: Telehealth Consultation | Total: ₹500.00 (PAID)
| +-----+
```
- **Test Case Result:** Handled 400 errors and generated valid 200 receipts. Status: **PASS**.

Experiment 08: Integration of Third-Party Services (Email, SMS, Payment)

- **Directory:** `Expt-08-ThirdParty-Services`
- **Detailed Manual:** `Expt-08-ThirdParty-Services/INPUT_OUTPUT.md`

8.1 Input Specification

- **Files:** `server.js`, `package.json`, `public/index.html` (Nodemailer, Twilio, Razorpay)
- **Execution Command:** `bash cd Expt-08-ThirdParty-Services && npm install && PORT=8000 node server.js`
- **Sample Payloads Injected:**
 - Email: `{ "patientEmail": "asmit@example.com", "patientName": "Asmit Jogdand", "doctorName": "Dr. Vidhate" }`
 - SMS: `{ "phone": "9876543210", "patientName": "Asmit Jogdand", "appointmentTime": "10:30 AM" }`
 - Payment: `{ "amount": 800, "patientName": "Asmit Jogdand", "consultationType": "Cardiology OPD" }`

8.2 Output Specification

- **Email Output (Nodemailer Ethereal Cloud):** `json { "success": true, "message": "Clinical email successfully dispatched via SMTP gateway!", "data": { "service": "Email Gateway (Nodemailer)", "previewUrl": "https://ethereal.email/message/ar4ogQLEqXPhQbfqar4ohcAYPlgYhv4sAAAAAe2NjK1QAU0xuebsZSfheeA", "status": "Delivered" } }`
- **SMS Output (Twilio Protocol):** `json { "success": true, "data": { "service": "SMS Gateway", "recipient": "9876543210", "sid": "SMrrismdhypn7k1267pp923", "otp": 334340, "status": "Sent / Delivered" } }`
- **Payment Verification Output:** Captured order with verified digital signature. Status: **PASS**.

Experiment 09: Hosting Website with Domain Registration Process

- **Directory:** `Expt-09-Hosting-Domain-Process`
- **Detailed Manual:** `Expt-09-Hosting-Domain-Process/INPUT_OUTPUT.md`

9.1 Input Specification

- **Files:** `vercel.json`, `netlify.toml`, `package.json`, `public/index.html`
- **Execution Command:** `bash cd Expt-09-Hosting-Domain-Process && chromium-browser public/index.html`
- **Domain & Zone Input Records (`healthpulse-care.org`):**
- Apex `A Record` $\rightarrow$ `76.76.21.21` (TTL 3600)
- Subdomain `CNAME` (`www`) $\rightarrow$ `cname.vercel-dns.com` (TTL 3600)
- Mail `MX Record` $\rightarrow$ `10 mail.healthpulse-care.org` (TTL 3600)
- Anti-spoof `TXT Record` $\rightarrow$ `v=spf1 include:_spf.google.com ~all` (TTL 3600)
- Nameservers $\rightarrow$ `ns1.vercel-dns.com`, `ns2.vercel-dns.com`

9.2 Output Specification

- **Simulated DNS Query Console Output (`dig`):** `$ dig +noall +answer healthpulse-care.org A healthpulse-care.org. 3600 IN A 76.76.21.21 $ dig +noall +answer healthpulse-care.org TXT healthpulse-care.org. 3600 IN TXT "v=spf1 include:_spf.google.com ~all"`
 - **Browser DNS Inspector:** Interactive table displaying status `Active & Propagated` for all 4 resource record types. Status: **PASS**.
-

Experiment 10: Integration of SSL Certificate in Web Application

- **Directory:** `Expt-10-SSL-Certificate`
- **Detailed Manual:** `Expt-10-SSL-Certificate/INPUT_OUTPUT.md`

10.1 Input Specification

- **Files:** `generate-ssl.sh`, `san.cnf`, `server-key.pem`, `server-cert.pem`, `server-https.js`, `nginx-ssl.conf`
- **Execution Command:**
`bash cd Expt-10-SSL-Certificate && bash generate-ssl.sh && npm install && node server-https.js`
- **Cryptographic Settings:**
 - Key Algorithm: RSA 2048-bit
 - Protocol Enforced: `TLSv1.2` / `TLSv1.3`
 - SANs: `localhost`, `healthpulse-care.local`, `127.0.0.1`
 - Ports: HTTPS `8443`, HTTP Redirector `8080`

10.2 Output Specification

- **HTTP 301 Permanent Redirection Output** (`curl -I http://localhost:8080/`): `HTTP/1.1 301 Moved Permanently`
`Location: https://localhost:8443/`
- **Encrypted Telemetry JSON Response** (`curl -k -s https://localhost:8443/api/ssl-info`): ``json HTTP/1.1 200 OK Strict-Transport-Security: max-age=31536000; includeSubDomains; preload X-Content-Type-Options: nosniff X-Frame-Options: SAMEORIGIN

```
{ "httpsEnabled": true, "tlsProtocol": "TLSv1.3", "cipherSuite": "TLS_AES_256_GCM_SHA384", "hstsActive": true, "keyLength": "2048 bits", "subject": { "commonName": "localhost", "organization": "HealthPulse Super-Specialty Clinic", "department": "Computer Engineering Dept, RAIT", "location": "Navi Mumbai, Maharashtra, India" }, "student": "Asmit Jogdand (25CE1051)", "timestamp": "2026-10-01T09:32:23.735Z" } `` - Test Case Result: TLS 1.3 encryption, automatic 301 redirection, and HSTS headers verified. Status: PASS.
```

Final Verification Summary

All 10 experiments for **Asmit Jogdand (25CE1051)** have been verified and documented with complete Input and Output specifications: - Source files, configuration manifests, and scripts recorded as Inputs. - Execution commands and CLI test parameters benchmarked. - UI layouts, terminal outputs, HTTP status codes, and JSON response bodies captured as Outputs. - In-memory fallbacks and validation edge cases tested.

Prepared for laboratory evaluation at Ramrao Adik Institute of Technology (RAIT), D Y Patil Deemed to be University, Navi Mumbai.